

Car NFT dApp — Test Case

Connect via Sepolia, register a VIN, mint & view the NFT

Background: Networks, Chain IDs & MetaMask

The dApp can talk to two different Ethereum networks, and MetaMask (the browser wallet extension) is what tells the frontend which one it is currently connected to. The frontend reads MetaMask's reported chain ID (a hex string) and looks up the correct deployed contract address for that network in frontend/src/ utils/contract_utils.js (the CONTRACT_ADDRESSES map).

Hardhat (local development network)

A local Ethereum node run inside the dev container via "npm run node" (hardhat node). It starts pre-funded test accounts and prints their private keys to the terminal — useful for importing a funded account into MetaMask for local testing without spending real or testnet ETH.

- Chain ID (decimal): 31337
- Chain ID (hex): 0x7a69
- RPC endpoint: http://127.0.0.1:8545
- Contract address env var: REACT_APP_SMART_CONTRACT_ADDRESS_LOCAL (auto-written by scripts/deploy.js on deploy)

Sepolia (public Ethereum testnet)

A public, shared Ethereum test network. Unlike Hardhat's local node, transactions here are real network transactions (paid with free testnet ETH from a faucet) and are visible to anyone on Sepolia Etherscan. This is the network used to test the dApp the way a real user would experience it, before any mainnet deployment.

- Chain ID (decimal): 11155111
- Chain ID (hex): 0xaa36a7
- Contract address env var: REACT_APP_SMART_CONTRACT_ADDRESS (set after "npm run deploy:sepolia")

MetaMask's role

MetaMask is the bridge between the browser-based frontend and whichever network it is currently pointed at. When the user clicks "Connect Wallet", the frontend calls eth_requestAccounts; MetaMask returns the connected account and its current chainId. If the user switches networks inside MetaMask (e.g. from Hardhat Local to Sepolia), MetaMask fires a chainChanged event, the frontend re-reads the chain ID, and resolves a different contract address from the CONTRACT_ADDRESSES map — so the same UI works against either network without a rebuild.

Chain ID cheat sheet

Network	Decimal	Hex	Env var
Hardhat (local)	31337	0x7a69	..._SMART_CONTRACT_ADDRESS_LOCAL
Sepolia (testnet)	11155111	0xaa36a7	..._SMART_CONTRACT_ADDRESS

Background: Dev Container

The repo ships a VS Code Dev Container (.devcontainer/devcontainer.json) so the whole team runs an identical, reproducible environment instead of relying on locally installed Node/tooling versions.

- Base image: mcr.microsoft.com/devcontainers/base:ubuntu-24.04
- Features installed: Node.js 22, GitHub CLI, common shell utilities (zsh as default shell)
- postCreateCommand: "cd frontend && npm install" — runs automatically the first time the container builds
- Forwarded ports: 3000 (React dev server — auto-opens in the browser) and 8545 (Hardhat node RPC)

Port 8545 forwarding matters specifically for this test case: MetaMask normally runs in a browser on the developer's host machine, outside the container. Forwarding 8545 out of the container is what lets a host-machine MetaMask reach the Hardhat node running inside it — without this, "Hardhat Local" would be unreachable from MetaMask even though the node is running fine inside the container.

Test Case: Register a VIN and Mint an NFT via Sepolia

Preconditions

- MetaMask extension installed in the browser, with the Sepolia test network selected (built into MetaMask by default).
- The connected MetaMask account holds Sepolia test ETH (from a faucet) to pay gas.
- The VinCidRegistry contract has already been deployed to Sepolia ("npm run deploy:sepolia"), and REACT_APP_SMART_CONTRACT_ADDRESS in frontend/.env.local (or the Vercel deployment) points at that address.
- REACT_APP_PINATA_JWT is set so the frontend can pin metadata to IPFS via Pinata.
- Frontend is running and reachable in the browser ("npm start" inside frontend/, or the deployed site).

Steps

1. Open the frontend in the browser.
2. Click "Connect Wallet". MetaMask prompts the user to approve the connection (eth_requestAccounts).
3. Approve the connection with Sepolia selected as the active network in MetaMask (chain ID 11155111 / 0xaa36a7). The frontend reads this chain ID and resolves the Sepolia contract address from CONTRACT_ADDRESSES.
4. Fill in the "Create NFT" form: VIN number, car brand, model, year, mileage, issue description, repair shop, and recipient wallet address.
5. Submit the form. The frontend validates the fields and the recipient address format client-side before doing anything on-chain.
6. The frontend builds a metadata JSON object from the form data and pins it to IPFS via Pinata's pinJSONToIPFS API, receiving back an IPFS content ID (CID).
7. The frontend calls storeCid(vin, cid, recipient) on the VinCidRegistry contract. MetaMask pops up asking the user to confirm and sign the transaction (gas paid in Sepolia ETH).
8. The transaction is broadcast and mined on Sepolia. The contract (an ERC-721) mints a new token to the recipient address and emits a CidStored(vin, cid, tokenId) event.
9. The frontend surfaces the resulting transaction hash and reports the NFT creation as successful.

Expected Result

- A new ERC-721 token exists on Sepolia, owned by the recipient address, linked to the submitted VIN and IPFS CID.
- The new entry appears in the frontend's "Show All Registered NFTs" list (reads getAllVins / getAllCidsAsList from the contract).
- The token is visible in MetaMask's NFT tab for the recipient account (may require "Import NFT" with the contract address and token ID if not auto-detected), and under Token Transfers for the contract on Sepolia Etherscan.
- Fetching the metadata by CID from the IPFS gateway returns the same VIN, make, model, mileage, etc. that was submitted.

Notes / Edge Cases

- If MetaMask is on a network other than Sepolia or Hardhat Local (any chain ID not in CONTRACT_ADDRESSES), the frontend has no contract address to use and logs "No contract address configured" — the create/read flows will fail.
- Switching networks mid-session (chainChanged) re-resolves the contract address without needing a page reload.
- On Sepolia, each step (approve connection, sign transaction) requires real user interaction with

MetaMask and may take 10–30s+ to mine, unlike the near-instant local Hardhat node.

